

B Production Deployment Guide

For the Engineering Department

Confidential — internal engineering document · Version 1.0 · July 2026

Read first — the single most important constraint. This application embeds its database (**PGLite**, Postgres compiled to WASM) with on-disk persistence in a local directory. It is therefore a **single-instance** application that requires a **persistent writable volume** and a **long-running Node server**. It is **not** compatible with serverless platforms (Vercel/Lambda), read-only filesystems, or horizontal auto-scaling as-is. To scale out, migrate the data layer to a managed Postgres (see §11). Plan the target platform around this before provisioning.

1 · Architecture at a glance

- **Framework:** Next.js 14 (App Router) + React 18 + TypeScript, run as a long-lived Node server (`next start`).
- **Data:** PGLite embedded Postgres persisted to a directory (default `.data/pg` , override with `PGLITE_DIR`).
Repositories speak plain parameterised SQL.
- **Market data:** all NGN Market API calls are proxied *server-side* through the app (the API key never reaches the browser), with an in-memory TTL cache in front of the upstream to conserve quota.
- **AI features:** Claude API (Anthropic SDK) powers the daily summary, filing extraction and portfolio review. Keys resolve DB-first (in-app, encrypted) then environment.
- **Background worker:** an in-process alert evaluator is started via Next's `instrumentationHook` and runs only in the Node runtime — another reason a persistent server (not serverless) is required.
- **Auth:** email/password (scrypt), httpOnly cookie sessions, RBAC (analyst < pm < admin), edge middleware page-gate.

2 • Prerequisites

REQUIREMENT	DETAIL
Node.js	v20 LTS or newer (the app is built and run on Node 20). Include <code>npm</code> .
Host	A Linux VM/container host (e.g. Ubuntu 22.04 LTS) with a persistent disk. ~1 vCPU / 1–2 GB RAM is sufficient for a single-firm workload; size up for build headroom.
Reverse proxy	nginx (or Caddy/Traefik) terminating TLS — required, because session cookies are <code>secure</code> in production.
Outbound network	HTTPS egress to <code>api.ngnmarket.com</code> , <code>doclib.ngxgroup.com</code> (filing PDFs) and the Anthropic API endpoint.
NGN Market API key	A live key (<code>ngm_...</code>). The server's egress IP may need to be allow-listed in the NGN Market dashboard.

3 • Environment variables

Provide these to the Node process (via your process manager, container env, or a root-owned `.env.local` that is **never** committed). **Required** values must be set before first boot.

VARIABLE	REQ?	PURPOSE
<code>NODE_ENV</code>	Yes	Set to <code>production</code> . Enables secure cookies and the encryption-key safety check.
<code>APP_ENCRYPTION_KEY</code>	Yes	Strong random secret (32+ chars) used to derive the AES-256-GCM key that encrypts stored API keys. In production the app refuses to start encrypting without it. Must be <i>stable</i> — rotating it makes previously saved keys unreadable (they must be re-entered).
<code>BRB_SEED_PASSWORD</code>	Yes	Password assigned to the three seeded staff accounts on first boot. Set a strong value; rotate the accounts after go-live (§6).
<code>NGNMARKET_API_KEY</code>	Optional*	NGN Market API key. *Optional here because it can be entered (encrypted) in-app under Admin → Settings; set it in env for unattended first boot.
<code>ANTHROPIC_API_KEY</code>	Optional	Claude API key for AI features. Can instead be entered in-app (encrypted). AI features degrade gracefully to a "not configured" notice when absent.
<code>PGLITE_DIR</code>	Recommended	Absolute path to the database directory on the persistent volume, e.g. <code>/var/lib/brb/pg</code> . Defaults to <code>.data/pg</code> under the app dir.
<code>NGNMARKET_BASE_URL</code>	Optional	Override the upstream base URL (defaults to the production v1 endpoint).
<code>PORT</code>	Optional	Server port (default 3000). Bind the reverse proxy to it.

Secrets handling. Store `APP_ENCRYPTION_KEY`, `BRB_SEED_PASSWORD` and any API keys in your secrets manager (or a `chmod 600`, root-owned env file) — never in the repo, image layers, or logs.

4 • Build & run

```
# 1. Fetch code and install exact dependencies
git clone <repo-url> brb-analyst && cd brb-analyst
npm ci

# 2. Quality gates (run in CI before promoting a build)
npm run typecheck
npm test
npm run lint

# 3. Production build
npm run build

# 4. Start the long-running server (bind to $PORT, default 3000)
NODE_ENV=production PORT=3000 npm run start
```

Run step 4 under a supervisor so it restarts on crash/reboot. Example **systemd** unit:

```
[Unit]
Description=BRB NGX Analyst
After=network.target

[Service]
WorkingDirectory=/opt/brb-analyst
EnvironmentFile=/etc/brb/brb.env          # holds the vars from $3 (chmod 600)
ExecStart=/usr/bin/node node_modules/.bin/next start -p 3000
Restart=always
RestartSec=5
User=brb
Group=brb

[Install]
WantedBy=multi-user.target
```

(PM2 is an equally valid supervisor: `pm2 start "npm run start" --name brb-analyst`.)

5 • Reverse proxy & TLS

Terminate TLS at nginx and proxy to the Node server. TLS is **mandatory** — the session cookie is issued with the `Secure` flag in production and will not persist over plain HTTP.

```
server {
    listen 443 ssl http2;
    server_name analyst.brbcapital.example;

    ssl_certificate      /etc/letsencrypt/live/analyst.brbcapital.example/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/analyst.brbcapital.example/privkey.pem;

    # Filing extraction reads multi-MB PDFs; allow a generous body + timeout.
    client_max_body_size 25m;
    proxy_read_timeout   300s;

    location / {
        proxy_pass          http://127.0.0.1:3000;
        proxy_http_version  1.1;
        proxy_set_header    Host                $host;
        proxy_set_header     X-Forwarded-For    $proxy_add_x_forwarded_for;
        proxy_set_header     X-Forwarded-Proto $scheme;
    }
}

server { listen 80; server_name analyst.brbcapital.example; return 301 https://$host$request_uri; }
```

Obtain certificates with certbot (`certbot --nginx`) and enable auto-renewal.

6 • First boot & account hardening

On first start the app creates its schema and seeds three staff accounts using `BRB_SEED_PASSWORD` :

EMAIL	ROLE
analyst@brb.local	analyst
pm@brb.local	pm (Principal Fund Manager)
admin@brb.local	admin

- 1 Sign in as `admin@brb.local` . Go to **Admin** → **Settings**.
- 2 Under **Users & roles**, create real staff accounts (Admin, PFM, Analyst) with strong passwords.
- 3 Delete or reset the seeded `@brb.local` demo accounts. (The platform prevents removing the last admin, so create your real admin first.)
- 4 Under **API keys**, enter the NGN Market and Claude keys if not supplied via env — they are encrypted at rest and take effect immediately.

7 • Data persistence & backup

- All application state (users, portfolios, ledgers, AI outputs, encrypted settings) lives in the PGLite directory (`PGLITE_DIR`). **This directory is the entire database — back it up.**
- Mount it on a persistent, snapshotted volume separate from the app code. In containers, use a named volume, not the image layer.

- Back up by snapshotting the volume or copying the directory **while the service is stopped** (or during a quiet window) to guarantee a consistent copy. Restore by replacing the directory and restarting.
- Keep a secure record of `APP_ENCRYPTION_KEY` : a restored data directory is undecryptable for its stored secrets without the same key.

8 • Security checklist

- ✓ `NODE_ENV=production` and TLS enforced end-to-end (secure cookies depend on it).
- ✓ `APP_ENCRYPTION_KEY` set to a strong, stable secret from the secrets manager (the app refuses weak fallbacks in production).
- ✓ Seeded demo accounts rotated/removed; `BRB_SEED_PASSWORD` is strong and not reused.
- ✓ Server egress IP allow-listed with NGN Market; firewall restricts inbound to 443 (and 22 from admin ranges only).
- ✓ Secrets never in the repo, image, or logs; env file is root-owned `chmod 600` .
- ✓ Data volume access restricted to the service user; backups encrypted at rest.

Known posture notes (from internal QA). (a) Auth is a same-site (`SameSite=Lax`) cookie with no separate CSRF token — acceptable with `Lax` + no cross-site mutating GETs, but keep cookies `Lax / Strict` and add an anti-CSRF token if you ever introduce embedded/cross-origin usage. (b) The edge middleware performs a cookie-*presence* page gate; real authorization is enforced in every API route and in server components via `getSession /role` checks — preserve that pattern for any new data-reading `page.tsx` . (c) Password resets revoke existing sessions; the "last admin" cannot be demoted/deleted.

9 • Observability

- **Logs:** the Node process logs to stdout/stderr — capture via `journalctl -u brb-analyst` (systemd) or `pm2 logs` , and ship to your log stack.
- **Health check:** point your load balancer/uptime monitor at `GET /login` (returns 200 without auth). Watch for restart loops.
- **API quota:** Admin → Quota & Audit surfaces NGN Market usage against the monthly pool and recent request latencies; the in-app audit trail records analyst actions.
- **Alerts worker:** confirm on boot that the instrumentation worker started (logged once) — it drives price/ex-dividend notifications.

10 • Upgrades & rollback

- 1 Build and test the new release in CI (`typecheck` , `test` , `build`) before promoting.
- 2 Snapshot the data volume.
- 3 Deploy: pull code, `npm ci && npm run build` , then restart the service. Schema changes are additive and idempotent (`CREATE TABLE IF NOT EXISTS`) and apply automatically on boot.
- 4 Smoke-test login, dashboard, and one portfolio. To roll back, restore the previous build and (if needed) the volume snapshot.

11 • Containerisation (optional) & scaling

Docker

A container is fine *provided* the data directory is a mounted volume and only one instance writes to it. Add `output: "standalone"` to `next.config.mjs` for a slim runtime image, then:

```
# Dockerfile (multi-stage sketch)
FROM node:20-slim AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-slim AS run
WORKDIR /app
ENV NODE_ENV=production PGLITE_DIR=/data/pg
COPY --from=build /app/.next/standalone ./
COPY --from=build /app/.next/static ./next/static
COPY --from=build /app/public ./public
VOLUME ["/data"]           # persistent DB volume – never bake into the image
EXPOSE 3000
CMD ["node", "server.js"]
```

Run a **single** replica with the volume attached. Do not scale this image horizontally.

Scaling beyond one instance

When one node is no longer enough, migrate off the embedded DB: stand up a managed Postgres, repoint the repository layer's `query()` (in `src/lib/db/client.ts`) at a `pg / postgres` connection pool — the SQL is standard and backend-agnostic — and export/import the existing data. The app can then run as multiple stateless replicas behind the load balancer, and the alert worker should be moved to a single dedicated worker instance to avoid duplicate evaluation.

12 • Troubleshooting

SYMPTOM	LIKELY CAUSE & FIX
Server exits on boot with an APP_ENCRYPTION_KEY error	Working as designed — set a strong APP_ENCRYPTION_KEY in production.
Login succeeds then immediately logs out	TLS not terminated / X-Forwarded-Proto not set → secure cookie dropped. Fix the proxy headers and serve over HTTPS.
Dashboard tiles show "API key not configured"	Set NGNMARKET_API_KEY (env) or enter it in Admin → Settings; confirm egress IP is allow-listed.
AI features show "not configured"	No Claude key resolvable — add it in Admin → Settings or set ANTHROPIC_API_KEY .
Data lost after redeploy	DB directory was not on a persistent volume — mount PGLITE_DIR on durable storage.
Filing extraction times out	Raise proxy_read_timeout and client_max_body_size at the proxy (see §5).